

TECHNICAL DOCUMENT

paper_trading 接入 Aris 认知体系 — 业务实施方案

Aris 认知体系 × paper_trading 全流程接入

wolf

2026-08-16

Contents

0. 现状盘点(先摸清家底)	3
0.1 paper_trading 已有的东西(复用,不重造)	3
0.2 Aris 认知体系已有的(接入基座)	3
0.3 关键结论(诚实基线)	4
1. 目标拆解:五个动词	4
2. 总体架构(目标态)	5
3. 分阶段实施方案	6
Phase 1:感知接入(学习 + 识别) — 先行	6
Phase 2:使用接入(使用) — 核心	7
Phase 3:管理闭环(管理) — 治理	8
4. 文件级改动清单	8
5. 验证方案(每阶段)	9
P1 验收	9
P2 验收	9
P3 验收	9
6. 风险与边界(诚实标注)	10
7. 决策点(需要用户确认)	10
8. 实施顺序建议	11

目标:让 Aris(数字生命体)能够自动学习、识别、采集、使用、管理 paper_trading 全业务流程,形成「交易认知闭环」。状态:规划稿(不实施),待确认后分阶段执行。日期:2026-08-16

0. 现状盘点(先摸清家底)

0.1 paper_trading 已有的东西(复用,不重造)

| 能力 | 位置 | 说明 | |—|—|—| | 数据模型 | laap/paper_trading/models.py |
Signal/Order/Trade/DecisionRecord/OutcomeRecord | | SQLite 存储 | data/paper_trading.db |
| 10 张表:signals/orders/trades/net_values/decisions/outcomes/evolutions/news_items/news_verdicts/
risk_rejections/news_summaries | | 决策闭环 | paper_service.py::PaperClosedLoop |
decide_and_trade → close_and_learn → settle → run_daily_cycle | | 交易自我 |
trading_self.py::TradingSelf | 人格 × 自我模型 → judge() 审核 → issue() 下达 | | 记忆桥 |
memory_bridge.py | 教训 encode_experience → UnifiedMemory,决策时检索注入 | | 参数进化 |
param_evolver.py + quant_evolution.py | 网络→随机→遗传,seed 固定可复现,LLM 增强 | | 新闻情
报 | news_pipeline.py / news_intel.py / news_verifier.py | 新闻采集+验证+摘要(哈希落盘) | | 真
实行情 | kline_source.py / market_source.py / data_sources.py | 腾讯 fqkline / akshare 800 天 |
| API 端点 | laap_brain/api.py /v1/quant/* | decisions/lessons/evolve/evolve_params/self/status/
daily_cycle/apply_params/evolve/approve|reject|audit/trades/net_values/signals/orders/outcomes | | 日
终调度 | daily_pipeline.py::QuantDailyScheduler | 交易日历 + 定时闭环 |

0.2 Aris 认知体系已有的(接入基座)

能力
规则引擎
aris_brain/aris_rules_engine.py + rules_tools.py
子串匹配,任务型触发,register_default_tools 注册工具
工具路由
laap/agi/tool_router.py
DSA 18 工具 11/11 命中
语义记忆
aris_brain/laap_semantic_memory.py
bge-small-zh 512 维,时间权重召回
PSI 意识

psi_jspace_bridge/
需求/情感/唤醒/循环
潜意识
aris_subconscious.py::QuantumSubconscious
V12.5 引擎,5s 后台线程
LLM 兜底
Agnes-2.5-flash(cpk- key)
开放话题

0.3 关键结论(诚实基线)

- 真实 A 股 OOS 回测通过率 10-20%,无泛化 **alpha**——系统当前定位是“受控模拟环境功能验证”
- 记忆桥(UnifiedMemory)和交易自我(TradingSelf)已存在,不是从零开始
- 用户立场:拒绝把负结果包装成“实证通过”

■ 1. 目标拆解:五个动词

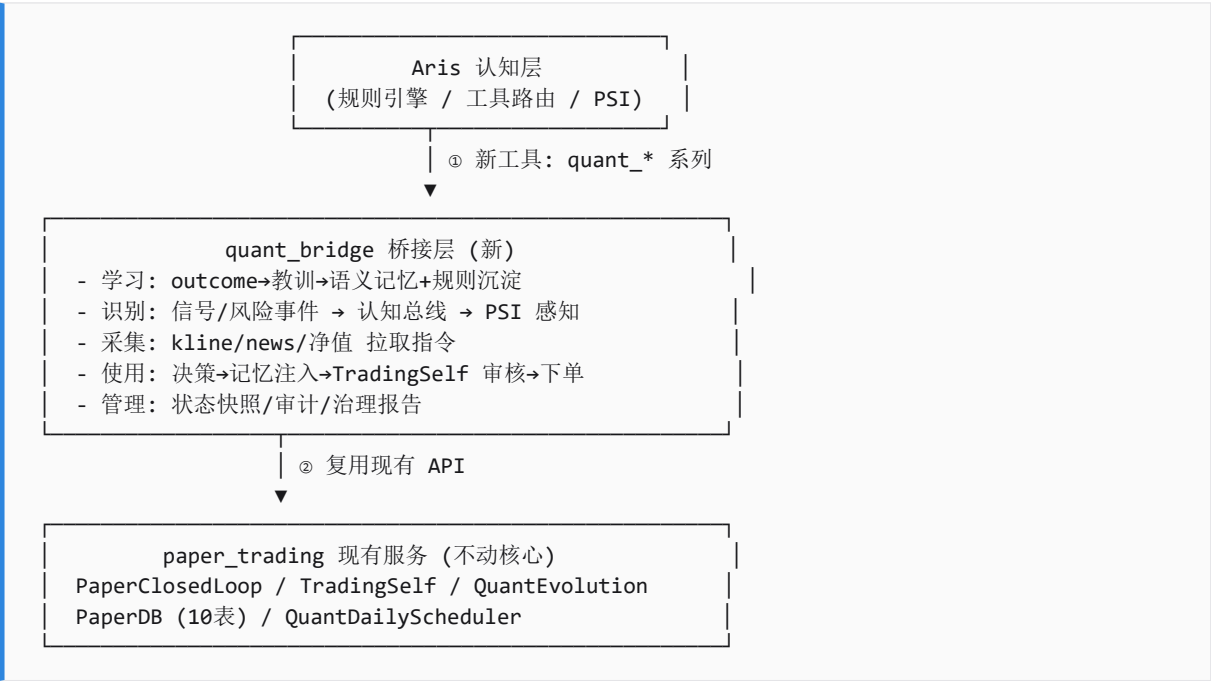
用户要求 Aris 对 paper_trading 全业务流程做到:

动词
学习
从每笔交易结果/教训中沉淀经验,形成可复用的交易认知
已有 memory_bridge 教训沉淀
教训只在 UnifiedMemory,未进 Aris 规则/人格
识别
识别行情/信号/风险/机会,并“意识到”它们
有行情采集,无 Aris 主动感知
需认知总线注入 PSI/意识
采集
主动拉取所需数据(行情/新闻/参数)
有被动采集(kline/news)

需 Aris 可发起的采集指令
使用
决策时调用工具/记忆/策略,产出交易决策
有 decide_and_trade
决策与 Aris 对话未打通
管理
监督全流程:状态监控、风险审核、进化治理、人工确认
有 TradingSelf.judge + 治理 API
无统一“管理视图”,Aris 无法主动报告

核心命题:让 Aris 从“被动回答交易问题”变成“交易业务的一等参与者”。

2. 总体架构(目标态)



设计原则(用户偏好:谨慎设计、不契约漂移):

- ✓ 最小改动:不动 paper_trading 核心契约(models/DB/服务签名)
- ✓ 复用既有语义:记忆桥/TradingSelf/治理 API 原样使用
- ✓ 新增独立桥接层:quant_bridge 是新增模块,不侵入现有代码
- ✓ 单向依赖:认知层 → 桥接层 → 服务层(服务层不知道 Aris 存在)

3. 分阶段实施方案

Phase 1:感知接入(学习 + 识别) - 先行

目标:Aris 能看到交易世界发生了什么,并沉淀为认知。

3.1 新增 Aris 工具(规则引擎侧,最小)

在 `aris_brain/rules_tools.py` 注册 4 个只读工具:

工具
<code>tool_quant_status</code>
交易状态/持仓/净值
GET /v1/quant/net_values + trades
<code>tool_quant_lessons</code>
交易教训/学到什么
GET /v1/quant/lessons
<code>tool_quant_portfolio</code>
持仓/仓位/风险
DB trades 未平仓行
<code>tool_quant_signals</code>
最近信号/交易信号
GET /v1/quant/signals

规则注册: `rules_defs.py` 加 `quant_status_rule` / `quant_lessons_rule` 等, pattern 覆盖口语变体(“我们赚了还是亏了”“最近交易怎么样”“有什么教训”)。

产出:用户问 Aris“最近交易怎么样”,Aris 能读 DB 真实回答。

3.2 认知总线事件注入(识别)

在 `cognitive_bus.py` 增加交易事件源:

事件类型: QUANT_SIGNAL / QUANT_TRADE_CLOSED / QUANT_RISK_TRIGGERED
QUANT_DAILY_SETTLE / QUANT_EVOLUTION_PROPOSED
→ 写入 cognitive_bus 事件队列 → Aris 下一轮对话感知
→ 同时推进 PSI: 盈利→valence+ / 亏损→valence-, 风险→arousal+

产出:平仓/止盈止损/风险触发时,Aris“心情”有真实变化,对话里会主动提到(“我注意到茅台那笔止损了,教训是...”)。

3.3 教训双写(学习增强)

现有 `encode_lesson` 已写 `UnifiedMemory`。新增第二写:把教训摘要写入 `laap_semantic_memory.json` (Aris 自己的记忆库),带标签【交易教训】+ 标的 + 类型,使 Aris 规则引擎的 `recall_fact_rule` 也能召回交易经验。

产出:问 Aris“记得交易上吃过什么亏吗”→能召回真实教训。

Phase 2:使用接入(使用) - 核心

目标:Aris 能发起完整交易动作,受 `TradingSelf` 审核。

3.4 新增 Aris 动作工具(带审核)

工具
<code>tool_quant_decide</code>
发起决策(buy/sell/hold + rationale)
走 <code>TradingSelf.judge()</code> 审核
<code>tool_quant_execute</code>
确认下单(审核通过后)
需 judge 通过 + 二次确认词
<code>tool_quant_close</code>
平仓
需 judge + 风险检查

契约:完全复用 `PaperClosedLoop.decide_and_trade` 签名, Aris 只提供 `(symbol, action, qty, rationale)`,不接触 DB。

触发规则:

- 用户说“帮我看下五粮液要不要买”→Aris 调 `tool_quant_decide` → judge 审核 → 返回建议
- 用户说“执行”→ `tool_quant_execute` → 下单成交

安全设计:

- 默认 `paper_trading_auto_execute=false` (配置),Aris 只给建议不自动下单
- 下单前必须用户明确确认(二次确认词)
- 全部操作写 `risk_rejections` 审计表(已有)

3.5 LLM 微调闭环(已有,接通)

`llm_refine.py` + `HermesIntegration.llm_call` 已存在——确保 `_get_llm_refine_fn()` 在服务启动时正常注入(检查依赖),使参数进化走 LLM 增强路径,Aris 可对进化提案给出“交易员视角”意见。

Phase 3:管理闭环(管理) - 治理

目标:Aris 成为交易的“监督者”,能主动报告和治理。

3.6 每日交易简报规则

`quant_daily_brief_rule`:触发词“今日交易简报/今天交易怎么样”,读 `net_values + outcomes + lessons`,生成结构化简报(净值/盈亏/教训/明日关注)。

产出:每天收盘后用户问一句,Aris 给完整复盘。

3.7 进化治理接入

复用 `/v1/quant/evolve/approve|reject|audit`:

- Aris 规则 `quant_evolution_rule`:触发词“进化提案/策略改进”,读 `audit` → 总结待批提案 → 用户决定 → 调 `approve/reject`

产出:用户说“看下有哪些进化提案”,Aris 列出并代用户批准/拒绝。

3.8 日终自动认知快照(cron)

新增 `cron(no_agent 脚本模式,复用 memorize_market_daily.py 模式)`: 每日 15:30 收盘后,脚本拉取 `quant` 状态 → 写入语义记忆 `【交易日报 YYYY-MM-DD】...` → Aris 下次对话自动感知。

产出:Aris 记得每天的交易情况,能跨日复盘(“昨天亏了,今天谨慎些”)。

4. 文件级改动清单

文件
<code>aris_brain/rules_tools.py</code>
+4~7 个 <code>tool_quant_*</code> 只读/动作工具
P1/P2
<code>aris_brain/rules_defs.py</code>
+3~5 条规则(<code>quant_status/lessons/brief/evolution</code>)
P1/P3
<code>aris_brain/cognitive_bus.py</code>
+交易事件源(5 类事件 → PSI 感知)
P1
<code>laap/paper_trading/memory_bridge.py</code>
<code>encode_lesson</code> 双写(UnifiedMemory + 语义记忆)

P1
laap_brain/api.py
确认 _get_llm_refine_fn 注入正常(可能微调)
P2
~/AppData/Local/hermes/scripts/
+ memorize_trading_daily.py cron 副本
P3
配置
+ paper_trading_auto_execute=false 等开关
P2

不动(契约漂移风险):models.py/db.py/paper_service.py 核心签名/trading_self.py/param_evolver.py/
现有 /v1/quant/* 端点行为。

5. 验证方案(每阶段)

P1 验收

1. Aris 对话: "最近交易怎么样" → engine: rules:quant_status, 内容含真实净值
 2. Aris 对话: "有什么交易教训" → 召回真实教训(含标的/类型)
 3. 模拟平仓 → 认知总线事件 → 下一轮 Aris 主动提及
 4. 语义记忆含 【交易教训】 条目, recall 可命中

P2 验收

1. "帮我看下600519要不要买" → Aris 给建议(带 judge 审核痕迹)
 2. 用户确认 → 下单成交 → decisions/orders 表有记录
 3. 无确认时 → 只建议不下单(安全)
 4. risk_rejections 有完整审计

P3 验收

1. "今日交易简报" → 结构化复盘(净值/盈亏/教训)
 2. "看下进化提案" → Aris 列提案 → 批准/拒绝生效
 3. cron 15:30 写入交易日报 → 次日 Aris 跨日感知

■ 6. 风险与边界(诚实标注)

风险
负 alpha 现实:策略本身不盈利,接入再漂亮也是管理“亏钱流程”
高
保持“模拟验证”定位,不承诺收益;Aris 如实汇报
自动下单失控
高
默认 auto_execute=false + 二次确认 + TradingSelf 审核
规则引擎子串匹配误触发交易工具
中
动作工具 require 明确触发词 + 审核,只读工具无副作用
认知总线事件刷屏(交易频繁时)
中
事件聚合(每类只保留最新 N 条)
cron 与主流程并发写 DB
低
SQLite WAL + 现有锁机制
NAS 同步覆盖本地规则引擎修改
中
及时 commit+push(已有教训)

■ 7. 决策点(需要用户确认)

1. 执行边界:P2 动作工具默认 `auto_execute=false` (Aris 只建议),还是允许自动下单?- 推荐:先 false 跑稳,观察 2 周再评估
2. 数据范围:接入哪些标的?当前 600519/000001/000858,还是扩展到全部自选股?- 推荐:保持现有 3 标的 + 已采集 K 线的标的
3. 认知注入强度:交易事件推进 PSI 到什么程度?- 推荐:只影响 valence/arousal(情绪),不动 competence/certainty(能力判断)
4. 简报频率:cron 每日一次(15:30)还是每周一次?- 推荐:每日(数据量小,便于跨日复盘)

■ 8. 实施顺序建议

P1 (感知)	→	P2 (使用)	→	P3 (管理)
2-3 天		3-4 天		2-3 天
(只读+记忆)		(动作+审核)		(简报+治理)

每阶段完成 → 验收 → 用户确认 → 进入下一阶段。Phase 1 不涉及资金动作,风险最低,建议先行。

本文档为规划稿,待用户确认后按阶段实施。设计原则:最小改动、复用既有契约、单向依赖、诚实定位。